Complete Explanation of Automatic Plant Watering System (Arduino Code for Beginners)

1 Introduction

This document provides a comprehensive explanation of an Arduino-based automatic plant watering system. The system integrates multiple hardware components and features a user-friendly menu interface. Below is a summary of the key components and their functions:

- **20×4 I2C LCD** – Displays system status, menus, and sensor readings.
- **4×4 Matrix Keypad** – Allows user input for navigation and settings.
- **DS3231 RTC Module** – Keeps accurate time for scheduled watering.
- **Soil Moisture Sensor** – Measures soil humidity to determine watering needs.
- **Pump with MOSFET Control** – Waters the plant when conditions require it.
- **Battery Voltage Divider** – Monitors power supply level.
- **EEPROM Storage** – Saves user settings permanently.

The system operates in two primary modes:

1. **Time-Based Mode** – Waters at a set time each day if soil is dry.
2. **Moisture-Based Mode** – Waters whenever soil becomes too dry (with cooldown).

We will now dissect the code section by section, explaining each part in simple terms suitable for beginners.

2 Libraries and Initialization

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <RTClib.h>
#include <EEPROM.h>

LiquidCrystal_I2C lcd(0x27, 20, 4);
RTC_DS3231 rtc;
```

- `Wire.h` – Enables I²C communication protocol (used by LCD and RTC).
- `LiquidCrystal_I2C.h` – Simplifies controlling an I²C LCD display.
- `Keypad.h` – Handles the matrix keypad scanning.
- `RTClib.h` – Provides functions for the DS3231 real-time clock.
- `EEPROM.h` – Allows reading/writing to Arduino's non-volatile memory.
- `lcd` – Object for the 20x4 LCD with I²C address 0x27.
- `rtc` – Object for the DS3231 RTC module.

3 Keypad Configuration

```
const byte ROWS = 4;
const byte COLS = 4;
char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS] = {A1, A2, 10, 2};
Keypad kp = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
```

- Defines a 4×4 keypad layout with characters for each button.
- `rowPins` and `colPins` map to Arduino digital/analog pins.
- `kp` object handles keypress detection and debouncing.

4 Pin Definitions and Calibration Constants

```
const int pumpPin = 3;          // PWM pin for pump (with MOSFET)
const int sensorPin = A3;       // Soil moisture sensor
const int batteryPin = A0;      // Battery voltage divider

const float V_REF = 5.0;        // Arduino supply voltage
const float R1 = 10000.0;       // Voltage divider R1 (10k)
const float R2 = 2200.0;        // Voltage divider R2 (2.2k)
const float BATT_MAX_V = 12.6;  // Li-ion 3S fully charged
const float BATT_MIN_V = 10.5;  // Li-ion 3S discharged
```

- `pumpPin` – PWM-capable pin for variable pump speed.
- `sensorPin` – Analog input for moisture sensor.
- `batteryPin` – Analog input for battery monitoring.
- Voltage divider calculations allow measuring up to 15V with 5V Arduino.
- Battery percentages are mapped between 10.5V (0%) and 12.6V (100%).

5 EEPROM Settings Storage

The system saves user preferences in EEPROM so they persist after power loss.

5.1 Settings Structure

```
struct Settings {
  uint16_t moisture;      // target moisture % (0-100)
  uint16_t hour;          // scheduled hour (0-23)
  uint16_t minute;        // scheduled minute (0-59)
  uint16_t pumpSeconds;   // pump run time in seconds (1-300)
  uint16_t checksum;      // XOR of all previous fields
};

Settings settings;
const Settings DEFAULT_SETTINGS = {50, 16, 0, 30, 0};
```

- `Settings` structure holds all adjustable parameters.
- `checksum` validates data integrity using XOR of all fields.
- `settings` is the active configuration.
- `DEFAULT_SETTINGS` provides safe initial values.

5.2 EEPROM Functions

```
uint16_t computeChecksum(const Settings& s) {
  return s.moisture ^ s.hour ^ s.minute ^ s.pumpSeconds;
}

void loadSettings() {
  EEPROM.get(EEPROM_SETTINGS_ADDR, settings);
  uint16_t calc = computeChecksum(settings);
  if (calc != settings.checksum) {
    settings = DEFAULT_SETTINGS;
    settings.checksum = computeChecksum(settings);
    saveSettings();
  }
}

void saveSettings() {
  settings.checksum = computeChecksum(settings);
  EEPROM.put(EEPROM_SETTINGS_ADDR, settings);
}
```

- `computeChecksum` calculates XOR of all data fields.
- `loadSettings` reads from EEPROM and validates checksum.
- If checksum fails, defaults are loaded and saved.
- `saveSettings` updates checksum before writing to EEPROM.

6 Global State Variables

```
int lastWateredDay = -1;          // for time-based irrigation(day stamp)
String inputBuffer = "";          // numeric input buffer

bool pumpRunning = false;
unsigned long pumpStartTime = 0;
unsigned long lastMoistureWatering = 0;
const unsigned long MOISTURE_COOLDOWN = 300000;  // 5 min

unsigned long lastInteractionTime = 0;
const unsigned long SCREEN_TIMEOUT = 60000;       // 60 seconds
bool screenOn = true;

bool timeMessageActive = false;
unsigned long timeMessageStart = 0;
const unsigned long TIME_MESSAGE_DURATION = 2000; // 2 seconds
```

- `lastWateredDay` – Prevents multiple waterings on same day.
- `inputBuffer` – Stores digits entered via keypad.
- `pumpRunning` – Indicates if pump is active.
- `lastMoistureWatering` – Implements cooldown for moisture mode.
- Screen saver variables – Turn off backlight after inactivity.
- `timeMessageActive` – Controls "Moisture OK" message display.

7 Menu State Machine

```
enum SystemState {
  MAIN_MENU,
  MODE_TIME,
  MODE_MOIST,
  SET_MOIST,
  SET_HOUR,
  SET_MIN,
  VIEW_BATT,
  SET_PUMP
};
SystemState currentState = MAIN_MENU;
```

The system uses a state machine with 8 distinct states:

- `MAIN_MENU` – Main navigation screen.
- `MODE_TIME` – Time-based irrigation mode display.
- `MODE_MOIST` – Moisture-based irrigation mode display.
- `SET_MOIST` – Adjust target moisture percentage.
- `SET_HOUR` – Set watering hour.
- `SET_MIN` – Set watering minute.
- `VIEW_BATT` – Display battery status.
- `SET_PUMP` – Adjust pump duration.

8 Core Helper Functions

8.1 Moisture Reading

```
int getMoisture() {
  int raw = analogRead(sensorPin);
  int m = map(raw, 1023, 200, 0, 100);
  return constrain(m, 0, 100);
}
```

Maps raw ADC value (0-1023) to percentage where:

- 1023 (dry air) → 0% moisture
- 200 (in water) → 100% moisture
- Result is constrained between 0-100%

8.2 Pump Control

```
void triggerPump() {
  pumpRunning = true;
  pumpStartTime = millis();

  // Wake screen
  lcd.backlight();
  screenOn = true;
  lastInteractionTime = millis();

  // Display message
  lcd.clear();
  lcd.setCursor(4, 1); lcd.print("PUMP␣ACTIVE");
  lcd.setCursor(3, 2); lcd.print("TIME:␣");
  lcd.print(settings.pumpSeconds);
  lcd.print("s");

  // Smooth start (ramp up PWM)
  for (int i = 0; i <= 255; i += 51) {
    analogWrite(pumpPin, i);
    delay(40);
  }
}

void stopPump() {
  analogWrite(pumpPin, 0);
  digitalWrite(pumpPin, LOW);
  pumpRunning = false;

  lcd.clear();
  lcd.setCursor(1, 1); lcd.print("WATERING␣FINISHED!");
  delay(1500);
  goBack();
}
```

- `triggerPump` – Activates pump with visual feedback and smooth start.
- `stopPump` – Turns off pump and returns to main menu.
- Smooth start gradually increases PWM to prevent mechanical stress.

8.3 Display Utilities

```
void printTime(int h, int m) {
  if (h < 10) lcd.print("0");
  lcd.print(h);
  lcd.print(":");
  if (m < 10) lcd.print("0");
  lcd.print(m);
}

void goBack() {
  currentState = MAIN_MENU;
  drawMainMenu();
}

void drawMainMenu() {
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("[A]TIME-M [B]MOIST-M");
  lcd.setCursor(0, 1); lcd.print("[C]SetTme [D]SetMst");
  lcd.setCursor(0, 2); lcd.print("[0]Pump Duration");
  lcd.setCursor(0, 3); lcd.print("[*]Battery [#]Exit");
}
```

- `printTime` – Formats time with leading zeros (e.g., 09:05).
- `goBack` – Returns to main menu and redraws it.
- `drawMainMenu` – Shows all available options with key mappings.

8.4 Input Handling

```
void handleNumberInput(char key) {
  if (key >= '0' && key <= '9' && inputBuffer.length() < 3) {
    inputBuffer += key;
  } else if (key == 'D' && inputBuffer.length() > 0) {
    inputBuffer.remove(inputBuffer.length() - 1);   // BACKSPACE
  }
}

void confirmSave() {
  lcd.clear();
  lcd.setCursor(4, 1); lcd.print("DATA SAVED!");
  delay(1000);
}
```

- `handleNumberInput` – Collects up to 3 digits; 'D' acts as backspace.
- `confirmSave` – Shows brief confirmation message.

9 Irrigation Logic

9.1 Time-Based Irrigation

```
void checkTimeIrrigation() {
  DateTime now = rtc.now();
  if (now.hour() == settings.hour && now.minute() == settings.
     minute && now.day() != lastWateredDay) {
    lastWateredDay = now.day();
    int moisture = getMoisture();
    if (moisture < settings.moisture) {
      triggerPump();
    } else {
      timeMessageActive = true;
      timeMessageStart = millis();
    }
  }
}
```

- Checks once per day at the scheduled time.
- Records that today's watering opportunity has been processed.
- If soil is dry → start pump.
- If soil is wet enough → show "Moisture OK" message.

9.2 Moisture-Based Irrigation

```
void checkMoistIrrigation() {
  if (lastMoistureWatering != 0 && (millis() - lastMoistureWatering
      < MOISTURE_COOLDOWN)) return;
  if (getMoisture() < settings.moisture) {
    triggerPump();
    lastMoistureWatering = millis();
    if (lastMoistureWatering == 0) lastMoistureWatering = 1;
  }
}
```

- Checks moisture level continuously.
- Enforces 5-minute cooldown between waterings.
- Starts pump when moisture drops below target.

10 Display Functions

Each display function updates its respective screen with rate limiting to avoid flicker.

10.1 Time Mode Display

```
void runTimeModeDisplay() {
  static unsigned long lastU = 0;
  if (millis() - lastU < 1000) return;
  lastU = millis();

  DateTime now = rtc.now();
  lcd.setCursor(0, 0); lcd.print("--- TIME CONTROL ---");
  lcd.setCursor(0, 1); lcd.print("Live Time: "); printTime(now.hour
      (), now.minute());
  lcd.setCursor(0, 2); lcd.print("Set Time : "); printTime(settings
      .hour, settings.minute);

  if (timeMessageActive) {
    if (millis() - timeMessageStart < TIME_MESSAGE_DURATION) {
      lcd.setCursor(0, 3); lcd.print("  Moisture OK!     ");
    } else {
      timeMessageActive = false;
      lcd.setCursor(0, 3); lcd.print("Waiting...   [#]Menu");
    }
  } else {
    lcd.setCursor(0, 3); lcd.print("Waiting...   [#]Menu");
  }
}
```

Updates every second, showing:

- Current time from RTC
- Scheduled watering time
- Status message ("Waiting..." or "Moisture OK!")

10.2 Moisture Mode Display

```
void runMoistModeDisplay() {
  static unsigned long lastU = 0;
  if (millis() - lastU < 500) return;
  lastU = millis();

  lcd.setCursor(0, 0); lcd.print("- MOISTURE CONTROL -");
  lcd.setCursor(0, 1); lcd.print("Live Moist: "); lcd.print(
      getMoisture()); lcd.print("%  ");
  lcd.setCursor(0, 2); lcd.print("Target    : "); lcd.print(
      settings.moisture); lcd.print("%  ");
  lcd.setCursor(0, 3); lcd.print("Monitoring.. [#]Menu");
}
```

Updates every 500ms, showing:

- Current moisture percentage
- Target moisture setting
- Status indication

10.3 Battery Display

```
void runBatteryDisplay() {
  static unsigned long lastU = 0;
  if (millis() - lastU < 1000) return;
  lastU = millis();

  float voltage=(analogRead(batteryPin)*V_REF/1024.0)*((R1+R2)/R2);
  int percent = constrain(map(voltage * 10, BATT_MIN_V * 10,
     BATT_MAX_V * 10, 0, 100), 0, 100);

  lcd.setCursor(0, 0); lcd.print("--BATTERY_STATUS--");
  lcd.setCursor(0, 1); lcd.print("Voltage:"); lcd.print(voltage, 2)
     ; lcd.print("␣V");
  lcd.setCursor(0, 2); lcd.print("Level:␣"); lcd.print(percent);
     lcd.print("%");
  lcd.setCursor(0, 3); lcd.print("Press␣[#]␣to␣Exit");
}
```

Calculates actual battery voltage using voltage divider formula and displays percentage.

10.4 Setting Screens

Three setting screens with similar structure:

- `runSetMoistDisplay` – Set target moisture percentage.
- `runSetPumpDisplay` – Set pump duration (1-300 seconds).
- `runSetTimeDisplay` – Set hour/minute for scheduled watering.

Each shows current value, input buffer with cursor, and available key commands.

11 Setup Function

```
void setup() {
  pinMode(pumpPin, OUTPUT);
  digitalWrite(pumpPin, LOW);

  lcd.init();
  lcd.backlight();

  if (!rtc.begin()) {
    lcd.print("RTC Error!");
    while (1);
  }

  // IMPORTANT: Set RTC to compile time ONCE, then comment out
  rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));

  loadSettings();

  // Splash screen
  lcd.clear();
  lcd.setCursor(3, 0); lcd.print("SAFE PLANT BY:");
  lcd.setCursor(1, 1); lcd.print("Jobaida  Rehnuma");
  lcd.setCursor(3, 2); lcd.print("Arman  Aditya");
  delay(2000);

  lastInteractionTime = millis();
  drawMainMenu();
}
```

- Initializes pump pin as output (off).
- Starts LCD and RTC (halts if RTC fails).
- **Critical:** Sets RTC to compile time. Should be commented after first use.
- Loads settings from EEPROM.
- Shows splash screen with team names.
- Draws main menu and initializes screen saver timer.

12 Main Loop

The loop handles four main tasks:

1. Pump timing when active
2. Keypad input and screen saver
3. State machine execution
4. Irrigation checks in appropriate modes

12.1 Pump Running State

```
if (pumpRunning) {
  if (millis() - pumpStartTime >= (settings.pumpSeconds * 1000UL)) {
    stopPump();
  }
  return;
}
```

When pump is running, only check if it should stop. No other processing occurs.

12.2 Screen Saver Logic

```
char key = kp.getKey();

if (key) {
  lastInteractionTime = millis();
  if (!screenOn) {
    lcd.backlight();
    screenOn = true;
    return;
  }
}

if (screenOn && (millis() - lastInteractionTime > SCREEN_TIMEOUT)) {
  lcd.noBacklight();
  screenOn = false;
}

if (!screenOn) return;
```

- Any keypress resets the inactivity timer.
- If screen was off, turn it on and ignore the key (prevents unintended actions).
- After timeout, turn backlight off.
- Skip rest of loop when screen is off.

12.3 State Machine Implementation

The switch statement handles all 8 possible states:

12.3.1 Main Menu State

```
case MAIN_MENU:
  if (key == 'A') { currentState = MODE_TIME; lcd.clear(); }
  else if (key == 'B') { currentState = MODE_MOIST; lcd.clear(); }
  else if (key == 'C') { currentState = SET_HOUR; inputBuffer = ""; lcd.clear(); }
  else if (key == 'D') { currentState = SET_MOIST; inputBuffer = ""; lcd.clear(); }
  else if (key == '0') { currentState = SET_PUMP; inputBuffer = ""; lcd.clear(); }
  else if (key == '*') { currentState = VIEW_BATT; lcd.clear(); }
  break;
```

Maps each key to a new state.

12.3.2 Mode States

```
case MODE_TIME:
  runTimeModeDisplay();
  checkTimeIrrigation();
  if (key == '#') goBack();
  break;

case MODE_MOIST:
  runMoistModeDisplay();
  checkMoistIrrigation();
  if (key == '#') goBack();
  break;

case VIEW_BATT:
  runBatteryDisplay();
  if (key == '#') goBack();
  break;
```

Each mode:

- Updates its display (with rate limiting)
- Performs irrigation checks
- Returns to main menu on '#'

12.3.3 Setting States

```
case SET_MOIST:
  runSetMoistDisplay();
  if (key == '*') {
    if (inputBuffer.length() > 0) {
      int val = inputBuffer.toInt();
      settings.moisture = constrain(val, 0, 100);
      saveSettings();
      confirmSave();
    }
    goBack();
  } else if (key == '#') {
    goBack();
  } else if (key) {
    handleNumberInput(key);
  }
  break;
```

Similar pattern for SET_PUMP.

12.3.4 Time Setting Sequence

```
case SET_HOUR:
case SET_MIN:
  runSetTimeDisplay(currentState == SET_HOUR);
  if (key == '*') {
    if (currentState == SET_HOUR) {
      if (inputBuffer.length() > 0) {
        int val = inputBuffer.toInt();
        settings.hour = constrain(val, 0, 23);
      }
      currentState = SET_MIN;
      inputBuffer = "";
      lcd.clear();
    } else { // SET_MIN
      if (inputBuffer.length() > 0) {
        int val = inputBuffer.toInt();
        settings.minute = constrain(val, 0, 59);
        saveSettings();
        confirmSave();
      }
      goBack();
    }
  } else if (key == '#') {
    goBack();
  } else if (key) {
    handleNumberInput(key);
  }
  break;
```

Two-step process:

1. Enter hour, press '*' to save and move to minute.
2. Enter minute, press '*' to save both and return.

13 Important Notes for Beginners

13.1 RTC Initialization

The line `rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));` sets the RTC to the date and time when the sketch was compiled. This should only be done once, then commented out. If left uncommented, the RTC will reset to the compile time every time the Arduino restarts.

13.2 Moisture Sensor Calibration

The mapping values (1023 → 0%, 200 → 100%) may need adjustment based on your specific sensor and soil type. Test your sensor in dry air and in water to get accurate min/max values.

13.3 Pump Control

The pump is controlled via PWM for smooth start. Ensure your MOSFET can handle the pump current and includes a flyback diode for protection.

13.4 Power Management

The screen saver extends battery life. The system continues monitoring even with screen off.

14 Summary

This code creates a fully functional automatic plant watering system with:

- Dual watering modes (time and moisture based)
- User-friendly menu navigation
- Persistent settings storage
- Battery monitoring
- Power-saving screen saver
- Error handling for RTC

The modular structure makes it easy to understand and modify for different requirements. Beginners can learn about:

- Arduino programming fundamentals

- State machine design
- EEPROM usage
- RTC integration
- Sensor reading and calibration
- Menu system implementation